# Statistical and Neural Approaches to 3D Femur Modeling

Boyer Timothé, Hacini Malik, Lainé Martin, Mouret Basile

January 23, 2026


# Data

- 24 scans of femurs
- 12 left and 12 right
- 18097 3D points corresponding to 54292 parameters

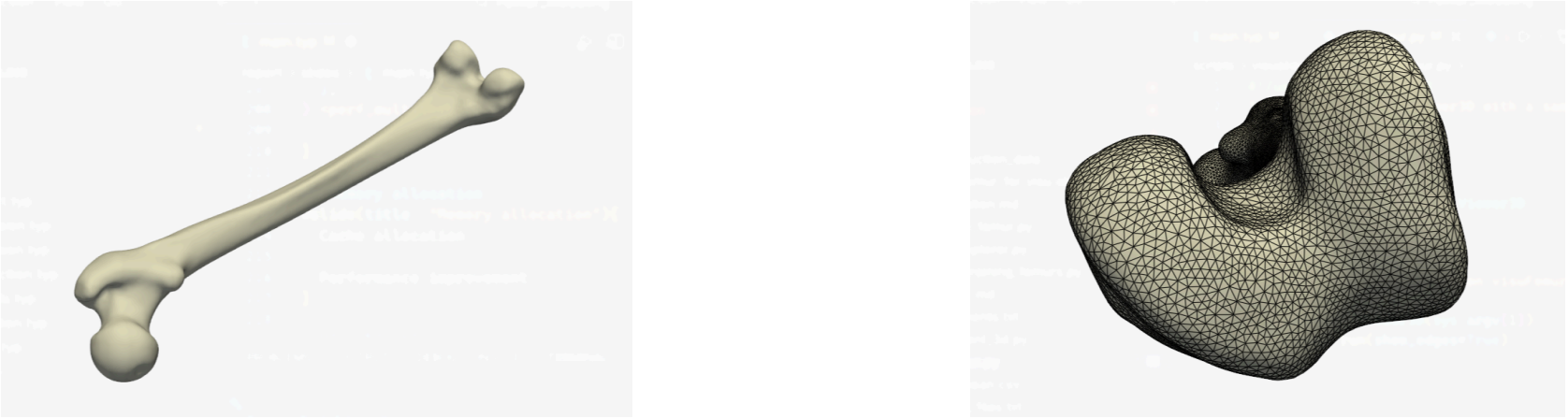


**Figure**: Example of a femur mesh

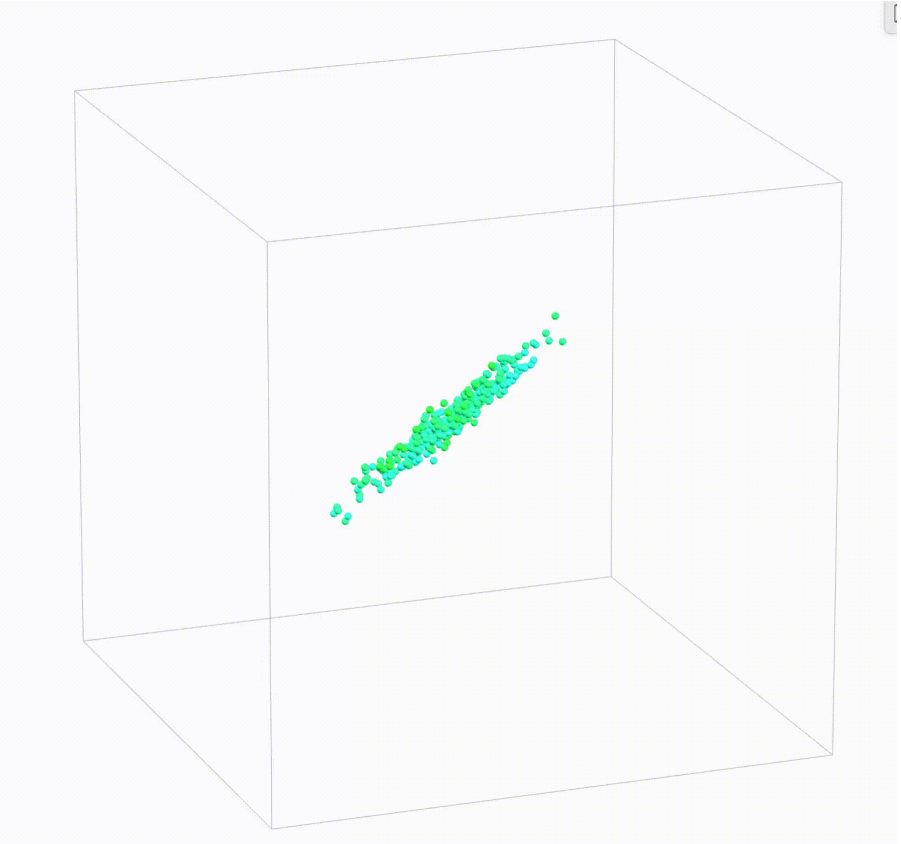
# What is there to learn ?

- *A priori*, femurs are *random* vectors of $\mathbb{R}^{3N}$
- In practice, femurs have structure : not any random $\mathbb{R}^{3N}$ vector represents an anatomically plausible femur !
  - Compact, distinct shaft and heads...

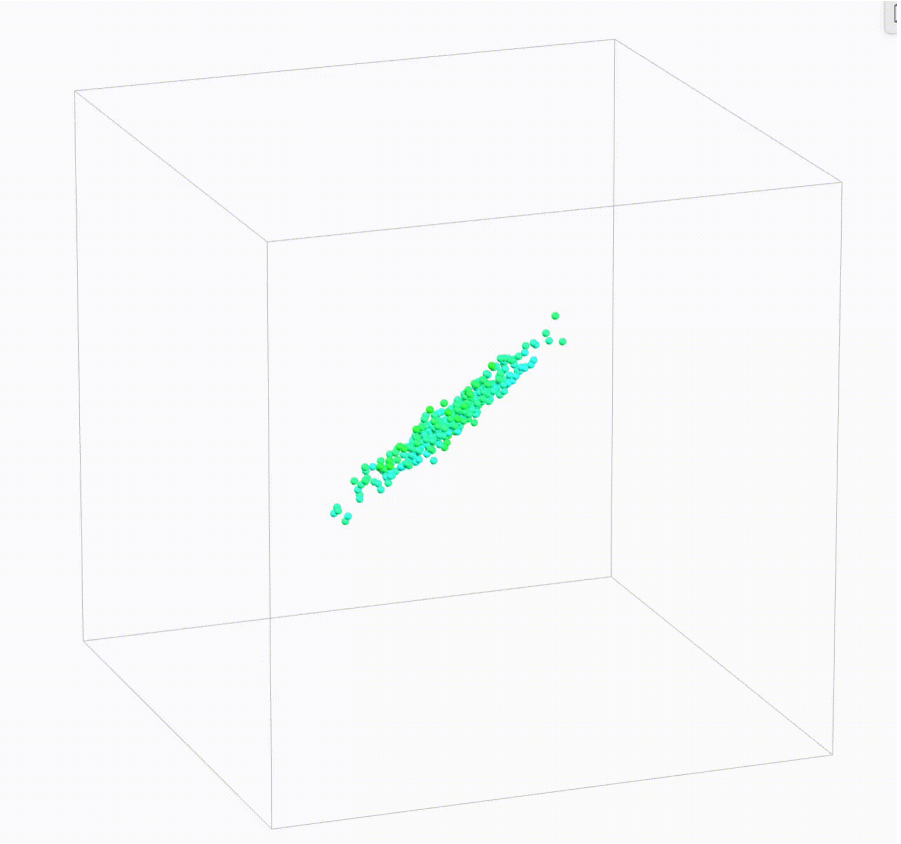
- Femur data thus lies in a low dimensional structure
  - Goal is learning this structure $\longrightarrow$ need for dimensionality reduction methods


**Figure**: Victor Wembanyama

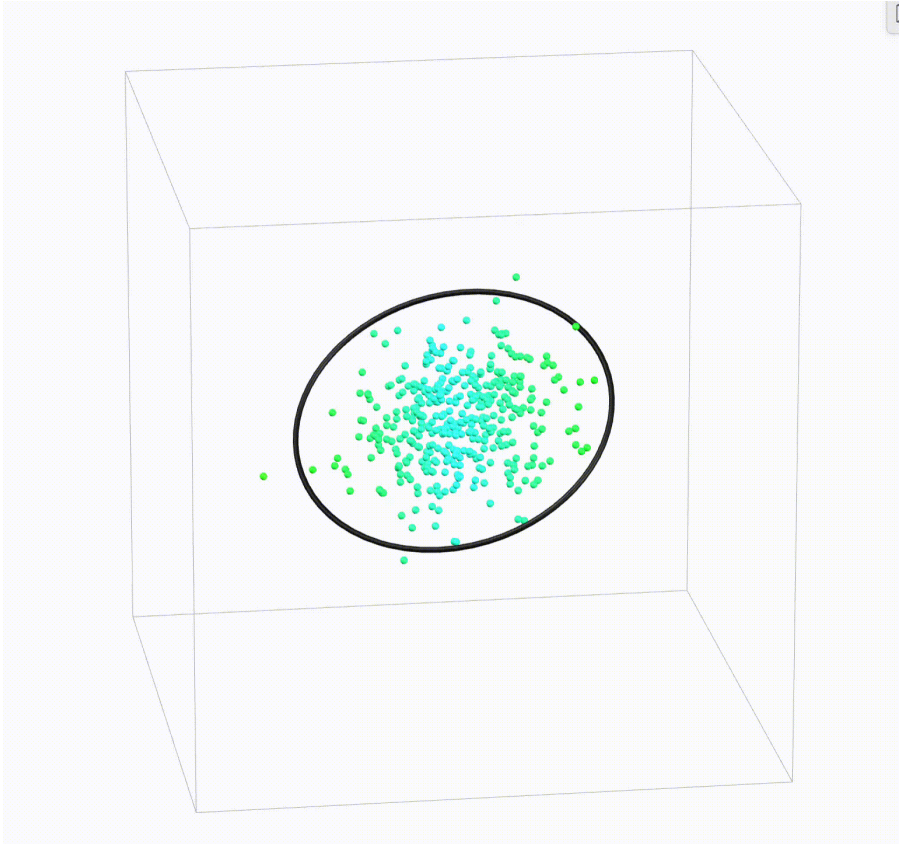
# Linear PCA : Principle



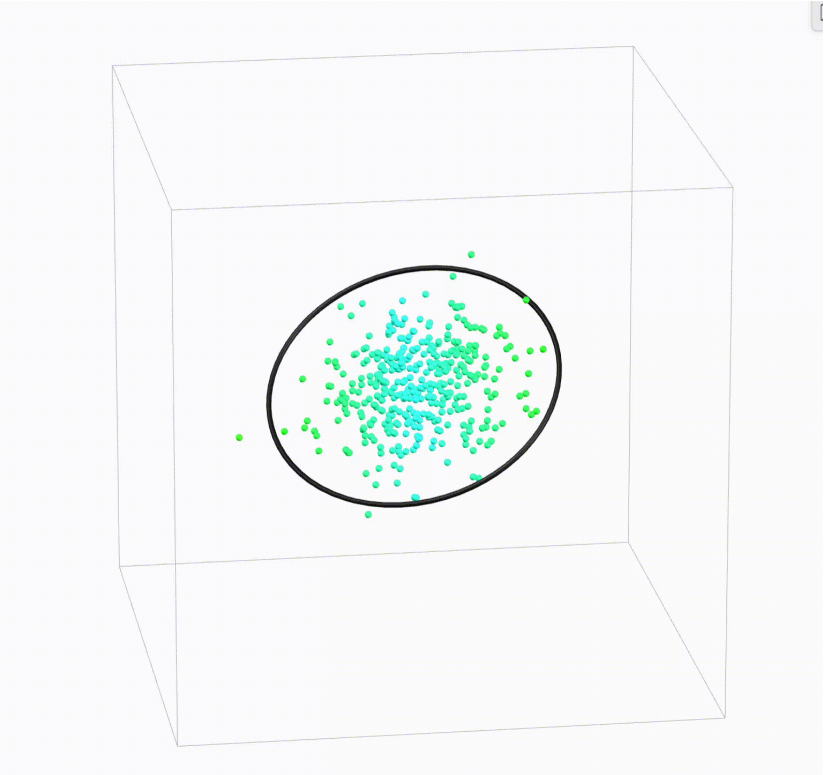
# Linear PCA : Principle



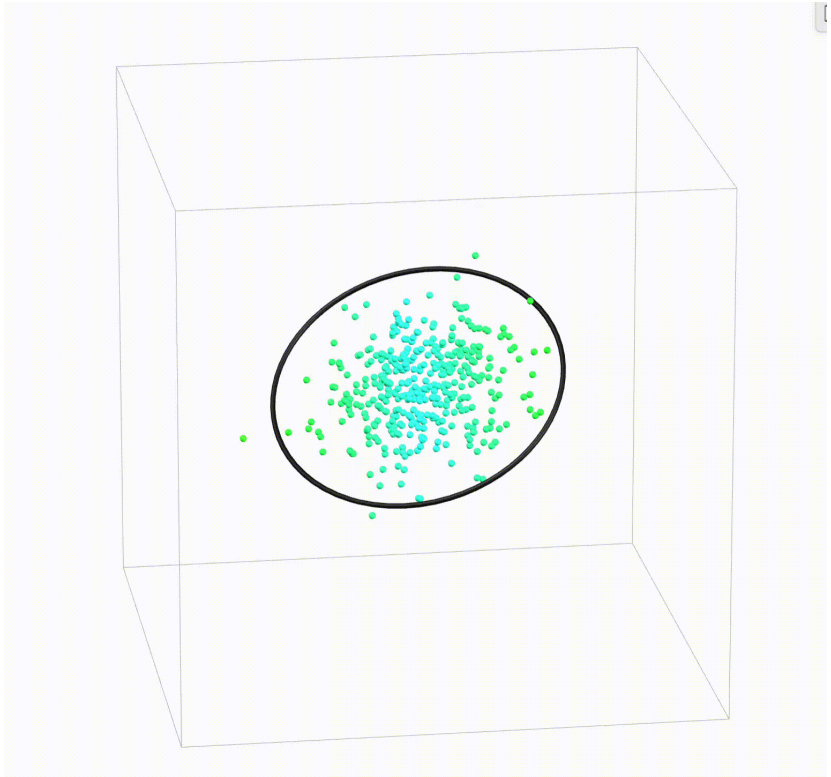
$\longrightarrow$
Ellipsoid fit



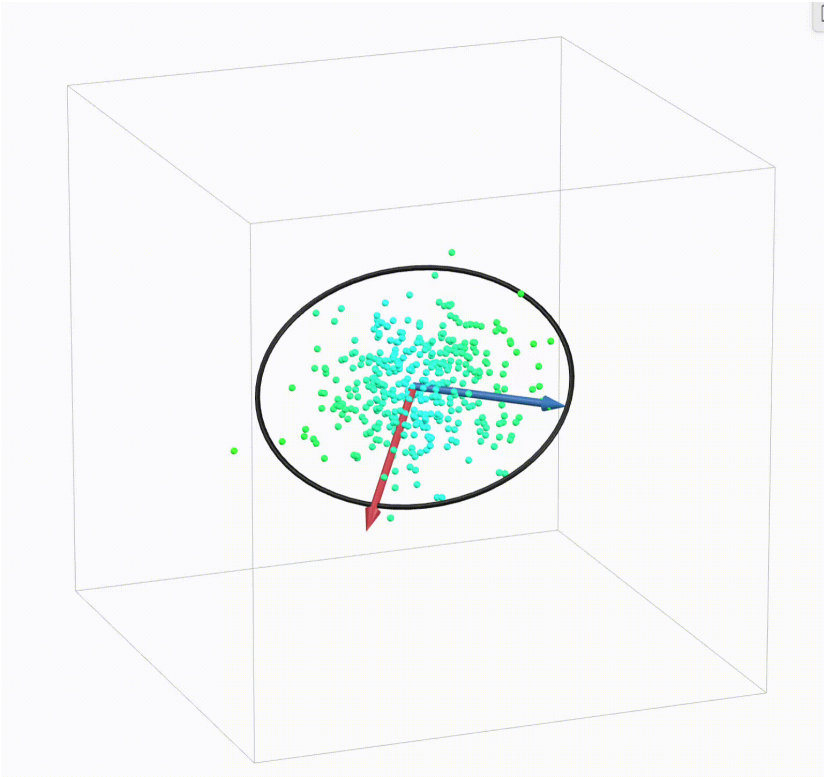
# Linear PCA : Principle



# Linear PCA : Principle



$\longrightarrow$
Eigendecomposition



# Linear PCA : A subtle potential issue

- PCA assumes the data is distributed as a gaussian point cloud
- If the data is made up of distinct clusters (e.g **healthy** and **unhealthy** femurs), the method breaks down.
- Luckily for us, the femur data originates from all types of individuals which averages out the cluster effect.

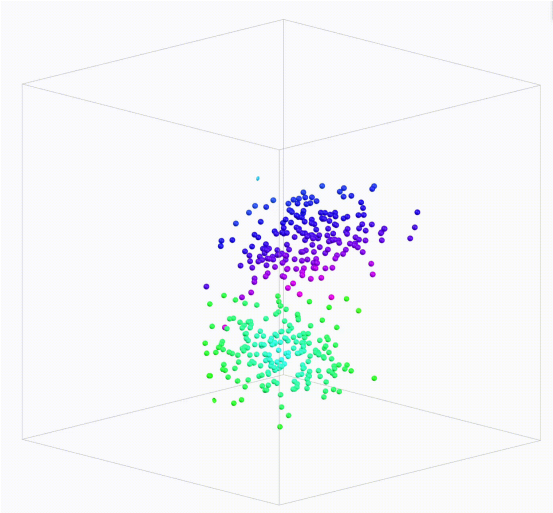


**Figure**: PCA is unadapted to datasets with highly clustered structure

# Linear PCA : Foundation

We represent each femur as a vector $S_i \in \mathbb{R}^{3P}$.

We compute the **Mean Femur** $\bar{S}$ and the sample covariance matrix $C$:

$$\bar{S} = \frac{1}{N}\sum_{i=1}^{N} S_i \quad , \quad S = \frac{1}{N-1}\sum_{i=1}^{N}(S_i - \bar{S})(S_i - \bar{S})^\top$$

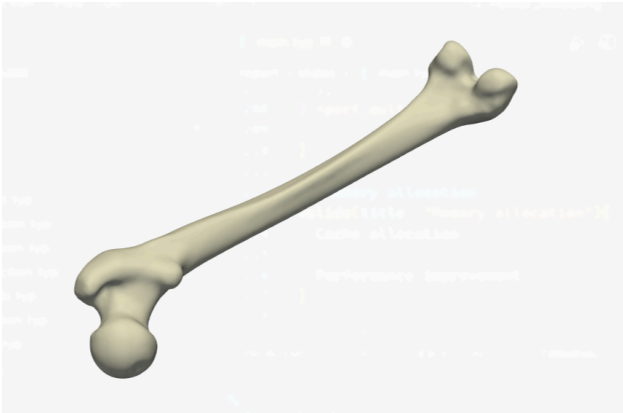


**Figure**: The (arithmetic) mean femur

# Linear PCA : Eigendecomposition

PCA orthodiagonalizes the covariance matrix to find the principal directions.

- The **eigenvectors** $v_k$ are the **Principal Components** (directions of variance).
- The **eigenvalues** $\lambda_k$ represent the variance captured by component $k$.
- Taking the $p$ top components effectively reduces the dimensionality of the dataset to $p$.

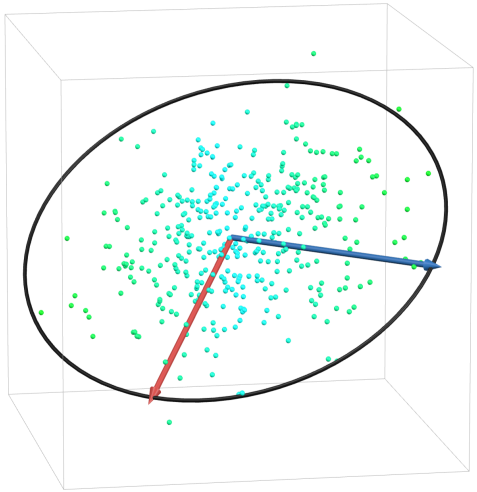


**Figure**: PCA reduces the data to it's $k$ most meaningful components

# Linear PCA Interpretation : Modes of Variation

Any femur instance $S$ in the dataset can be approximated as the mean shape plus a weighted sum of the principal components:

$$S \approx \bar{S} + \sum_{k=1}^{K} \omega_k v_k$$

- $\omega_k$: The standard deviation specific to this individual w.r.t component $k$.

  We can then visualize all possible linear deformations and try to interpret them clinically !

# Linear PCA : Results and Limitations

PCA assumes that the shape space is flat (a linear subspace of $\mathbb{R}^{3N}$).

- No reason to believe all linear transformations are anatomically plausible
- True anatomical deformations can be non-linear (e.g. torsion or bending).

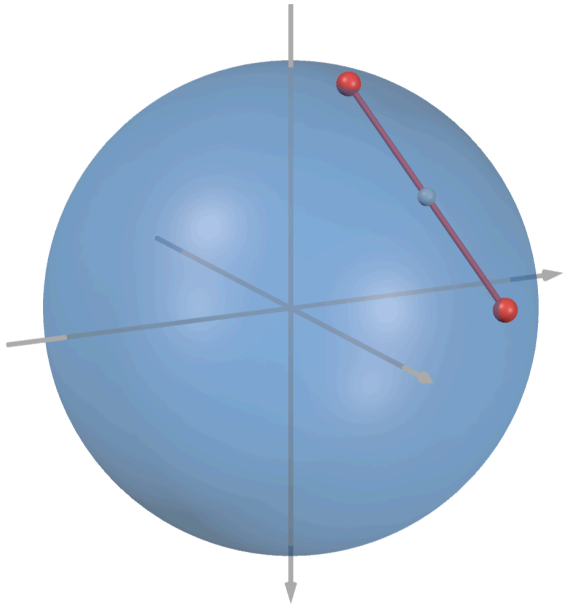


**Figure**: Eculidean transformations of points do not preserve underlying structure

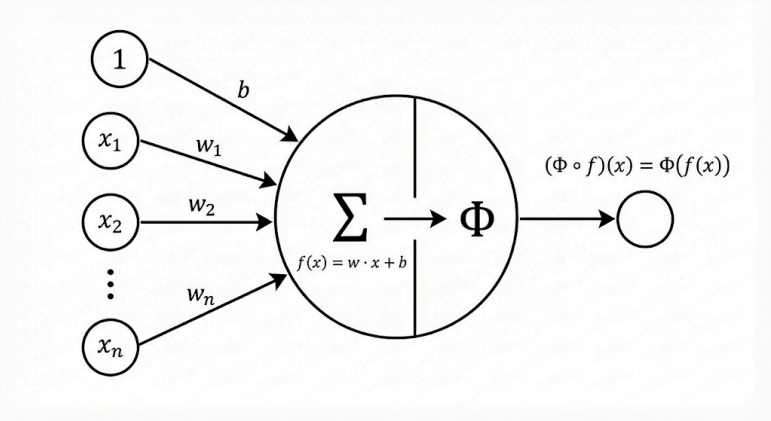
# Neural network

## Neuron structure

Given an input $x \in \mathbb{R}^n$

The characteristics of a neuron are:

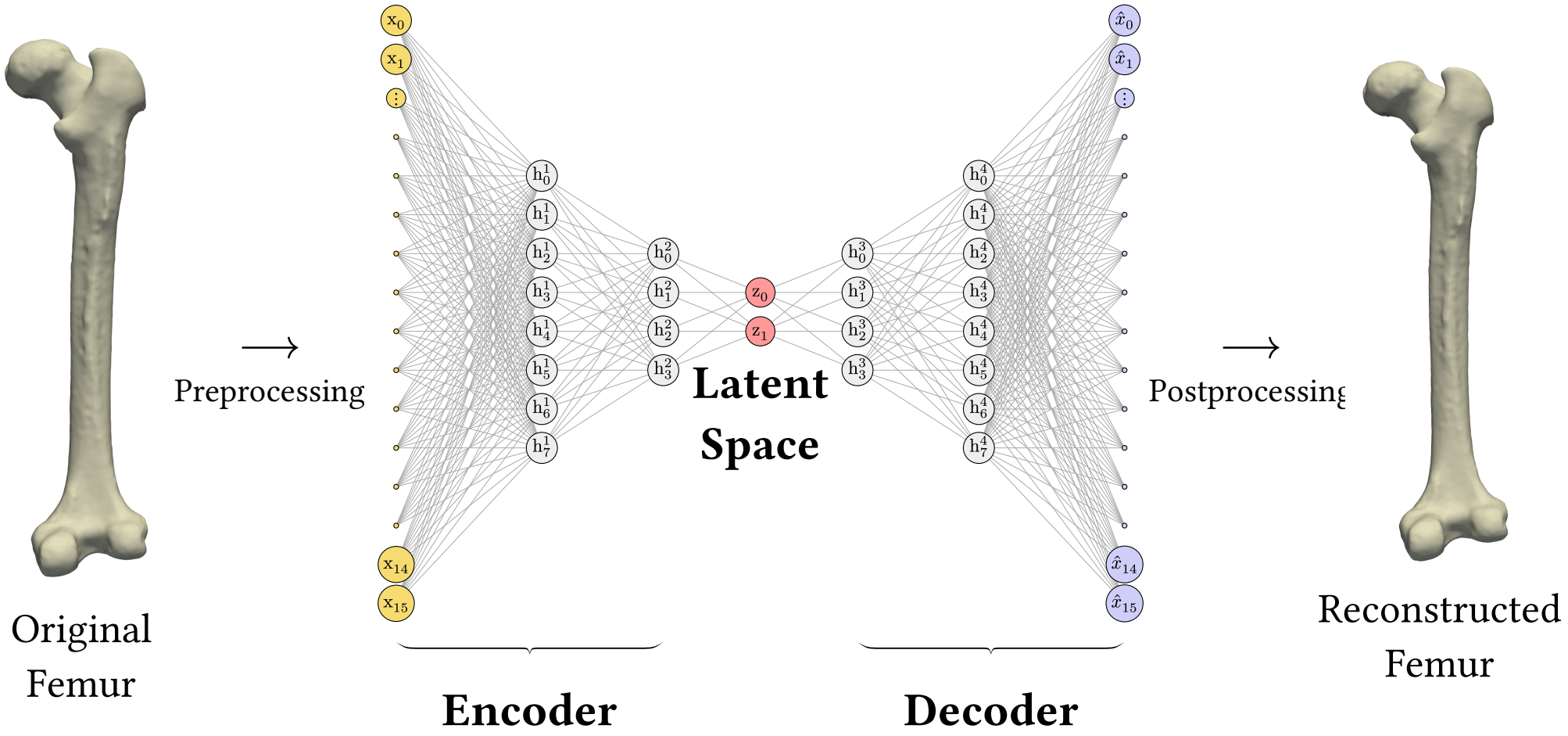
- Weights vector $w = (w_1, w_2, ..., w_n)$
- Bias term $b$
- Weighted sum function $f(x) = w.x + b$
- Activation function $\Phi$




## Output of a neuron:

$$(\Phi \circ f)(x) = \Phi(f(x))$$

# Neural network



# Training process

## First Model

- **Layers**: {54873, 1024, 256, 32, **10**, 32, 256, 1024, 54873}
- **Activation function** : Sigmoid
- **Loss function** : MSE
- **Preprocessing** : MinMax Normalization for each coordinate
- **Training** : 1000 epochs

# Training process

## First Model

- **Layers**: {54873, 1024, 256, 32, **10**, 32, 256, 1024, 54873}
- **Activation function** : Sigmoid
- **Loss function** : MSE
- **Preprocessing** : MinMax Normalization for each coordinate
- **Training** : 1000 epochs

## Problems

- Slow to train (> 100M parameters $\approx$ 500MB in memory)
- Vanishing gradient
- Loss distances aren't proportional to femur distance
- Boxed output

# Training Process

## Solutions

- Use a **linear output layer** so the model can take every value
- Change the activation function for **Tanh** and **LeakyReLU**
- Reduce the layer sizes
- Preprocessing : remove the **mean femur** and normalizing all coordinates **equally**

# Training Process

## Solutions

- Use a **linear output layer** so the model can take every value
- Change the activation function for **Tanh** and **LeakyReLU**
- Reduce the layer sizes
- Preprocessing : remove the **mean femur** and normalizing all coordinates **equally**

## Latest Models

- **Layers**: {54873, 1024, 256, 32, **10**, 32, 256, 1024, 54873}
- **better activation functions**: tanh and LeakyReLU
- **Linear** last layer
- New **Preprocessing**
- Longer **training**: 5000 epochs

# Linear algebra implementation

## Custom library design

- Built on top of **Eigen** for efficient internal storage
- Template-based classes
- Supports multiple numeric types: `float`, `double`, `int`, `long`, etc.

# Linear algebra implementation

## Custom library design

- Built on top of **Eigen** for efficient internal storage
- Template-based classes
- Supports multiple numeric types: `float`, `double`, `int`, `long`, etc.

## Main classes

### Vector

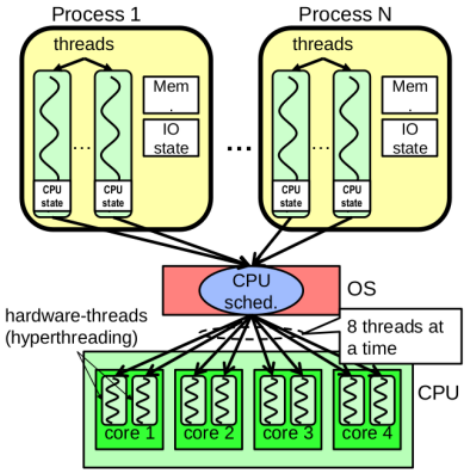
- Scalar/dot product
- Hadamard product
- Outer product

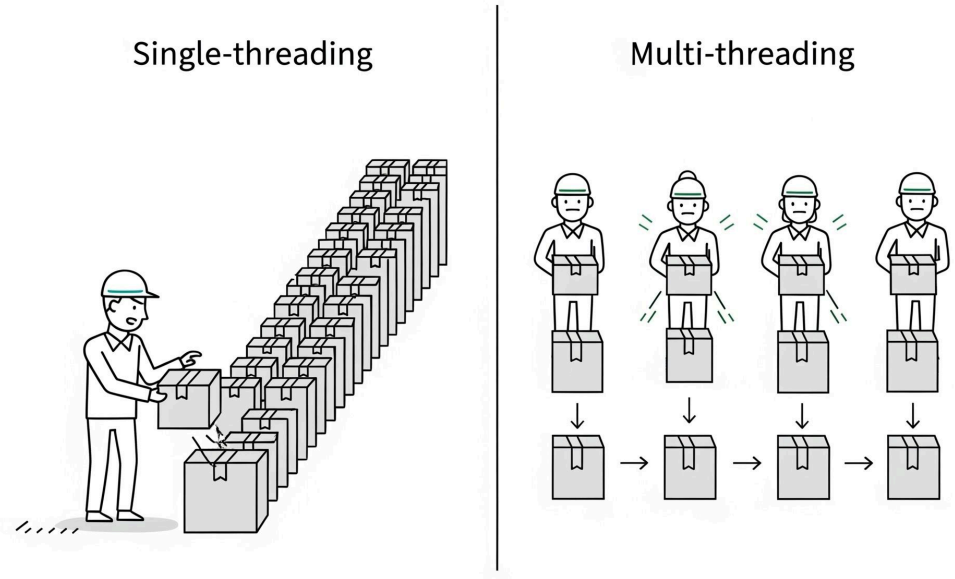
### Matrix2D

- Matrix multiplication
- Transpose
- Row/column extraction

# Motivation of multithreading

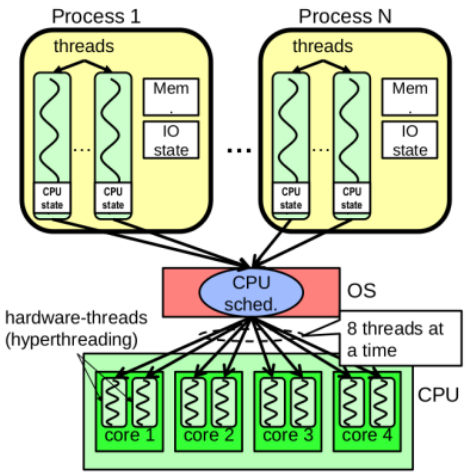
- Speed up the training process by using **multi-core processors**.

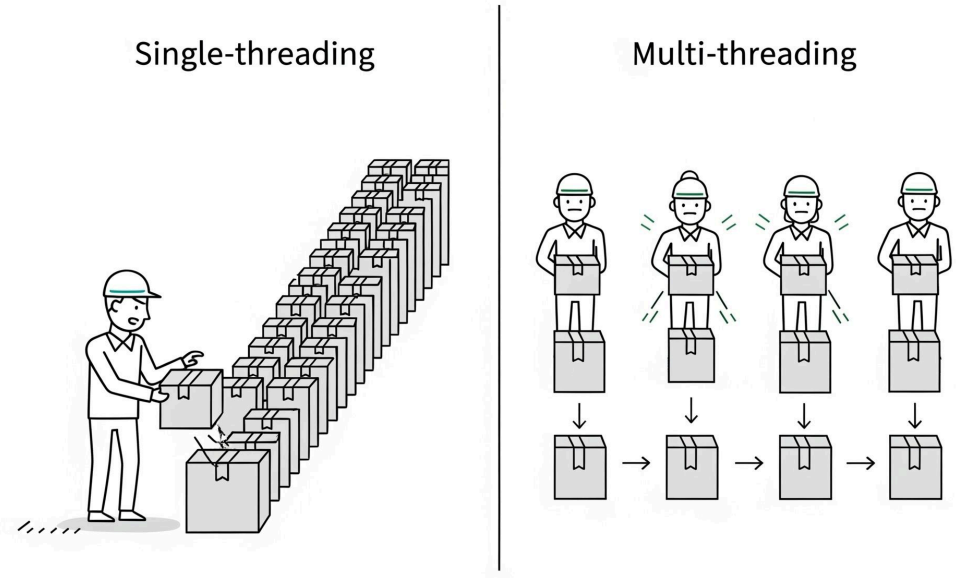


# Motivation of multithreading

- Speed up the training process by using **multi-core processors**.




- For **Matrix** $\times$ **Vector** multiplication :

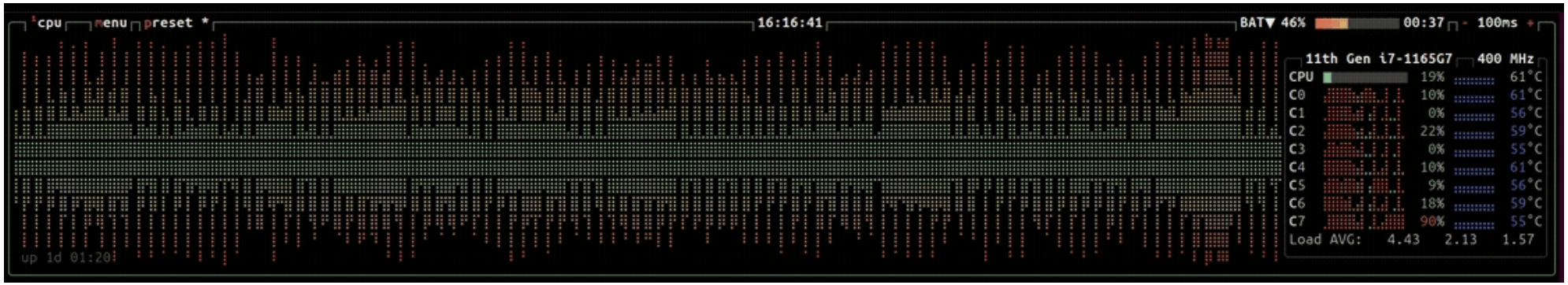
$$\begin{pmatrix} w_{11} & w_{22} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \dots & \dots & \dots & \dots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{pmatrix} \times \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{pmatrix}$$

# Multithreading with `std::thread`

1. For **every** parallelizable operation, create a thread
   −> naive implementation : time spent creating and destroying threads is **larger** than the time saved by parallelization !

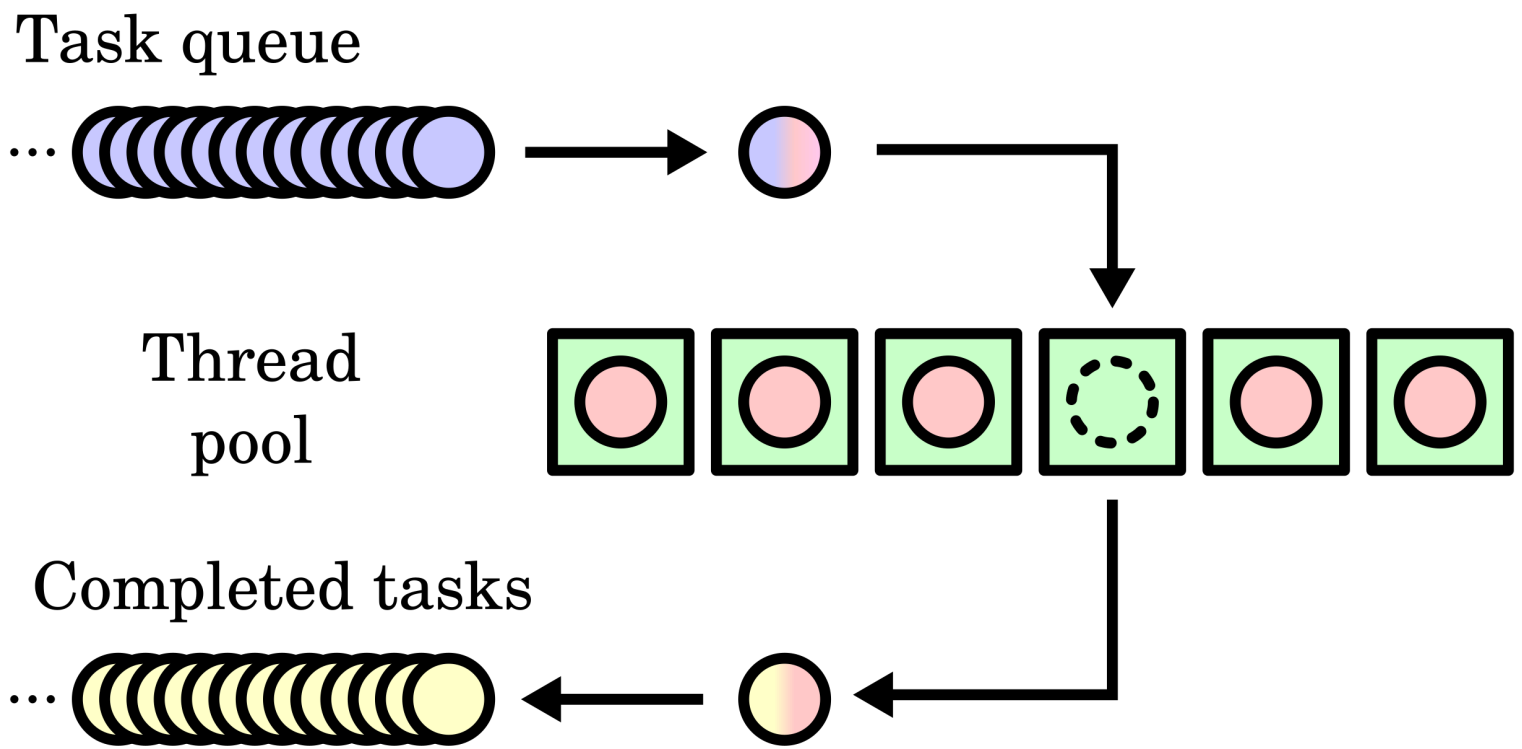
# Multithreading with `std::thread`

1. For **every** parallelizable operation, create a thread
   –> naive implementation : time spent creating and destroying threads is **larger** than the time saved by parallelization !
2. Create a **fixed** number of threads at the beginning of the function
   –> better, but still a lot of thread creation/destruction !



# Multithreading with `std::thread`

3. **Thread pool**



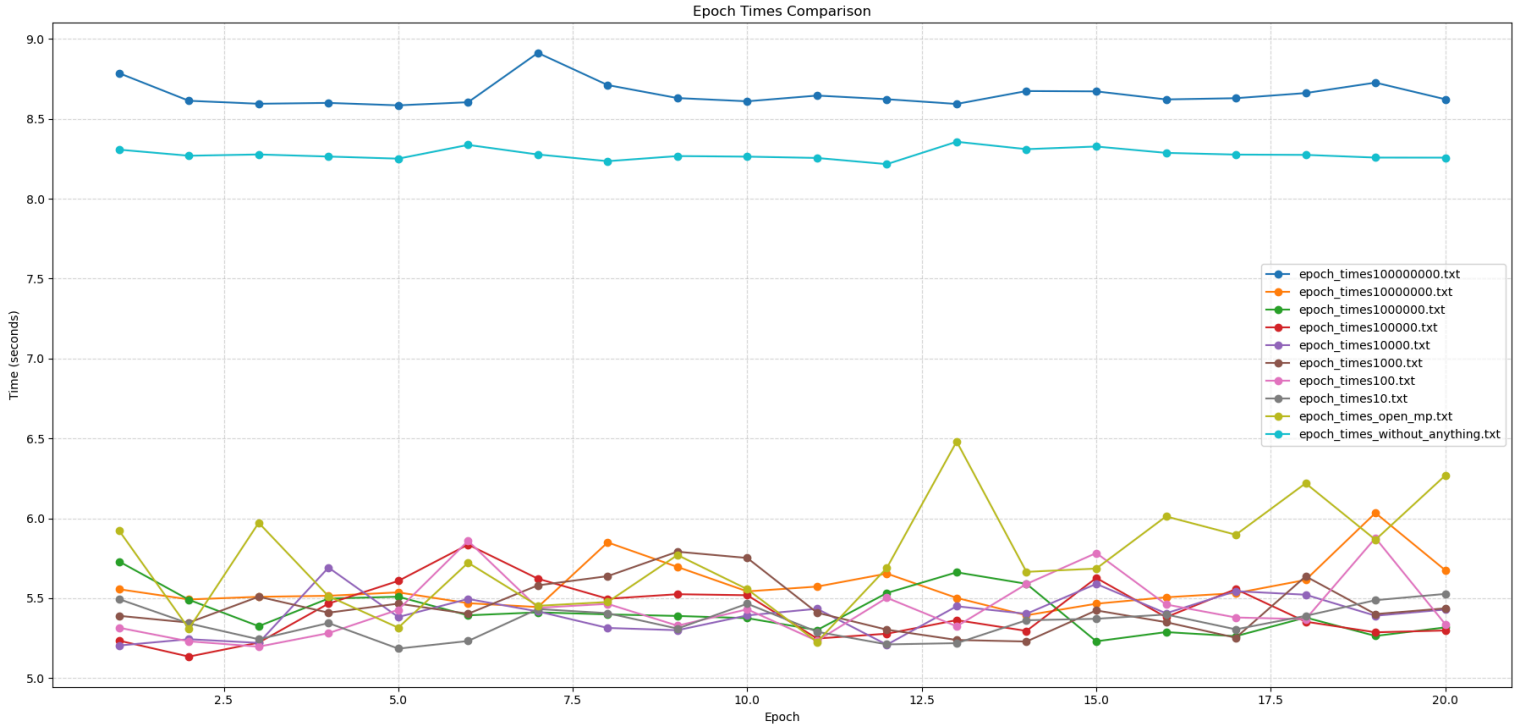
# OpenMP Multithreading

- **Automatically** manages thread creation and workload distribution
- **Simple to implement** (near to sequential code)

```
omp_set_num_threads(omp_get_max_threads());
#pragma omp parallel for

for (size_t j = 0; j < m_rows; ++j) {
    T sum = 0;
    for (size_t i = 0; i < m_cols; ++i) {
        sum += m_data(j, i) * vec(i);
    }
    result(j) = sum;
}
return result;
```

# Performance Graph

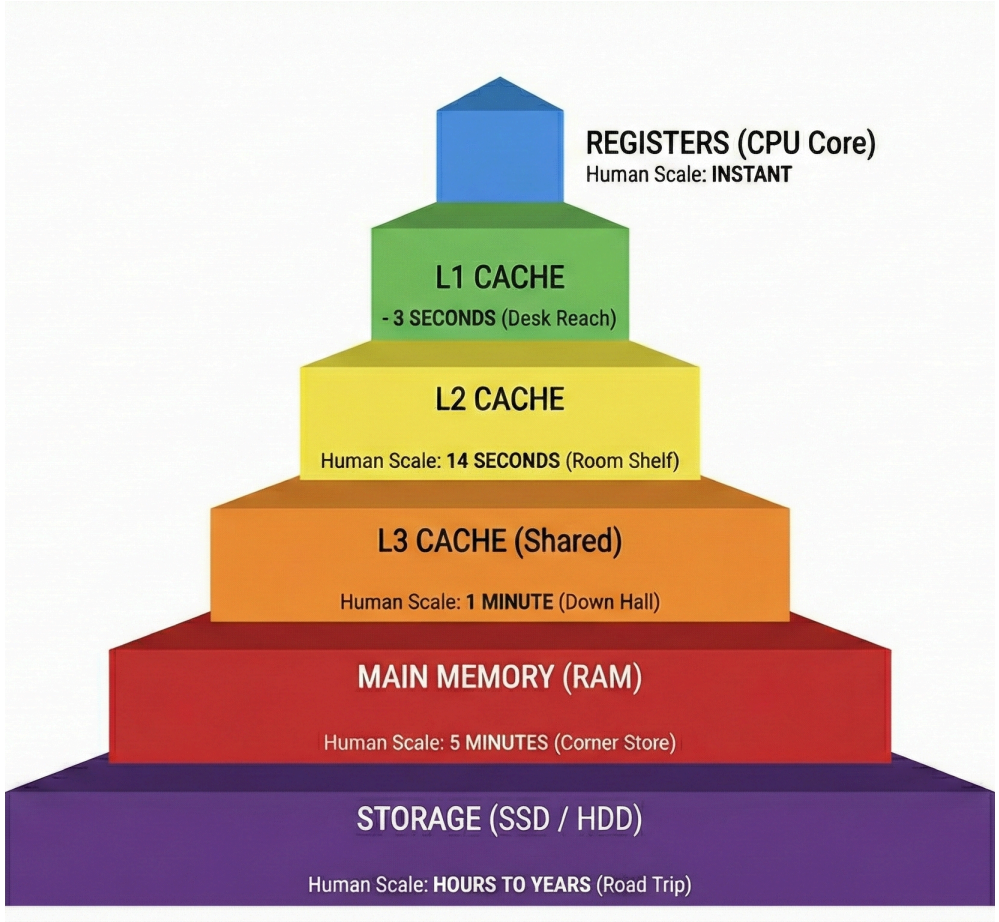



- Treshold parameter = number of parameters in the weitght matrix above which we use multithreading.

# Memory allocation

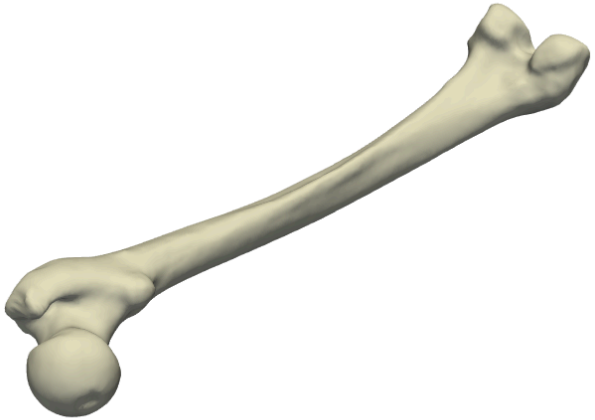
## Data Oriented Design

- improve **cache locality** by switching rows and columns acces (**$\times$ 2 speedup**)
- **Preallocation** of variables and Memory optimized functions (**$\times$ 4 speedup**)
- In total $\Longrightarrow$ **$\times$ 8 speedup**, going from 58 seconds to 7 seconds per epoch



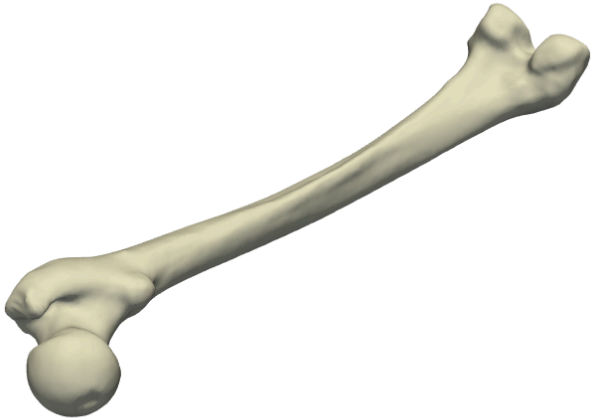
# First visualizations

## `visuFemur.py`

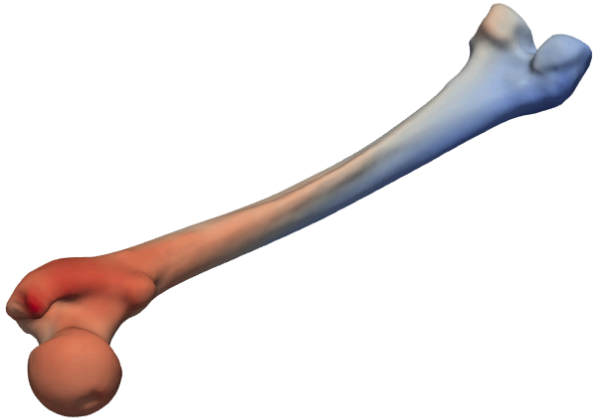


# First visualizations

## `visuFemur.py`

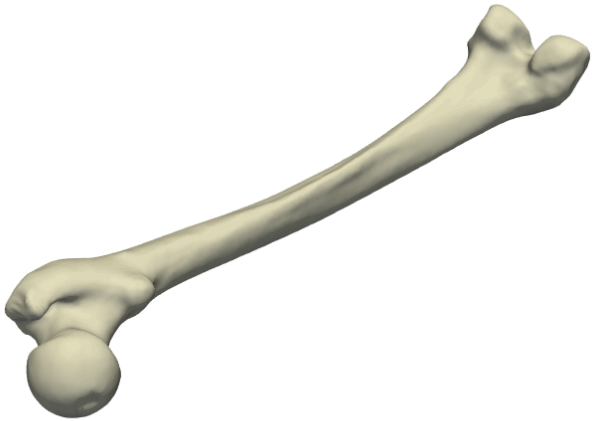


## `compare_femur.py`

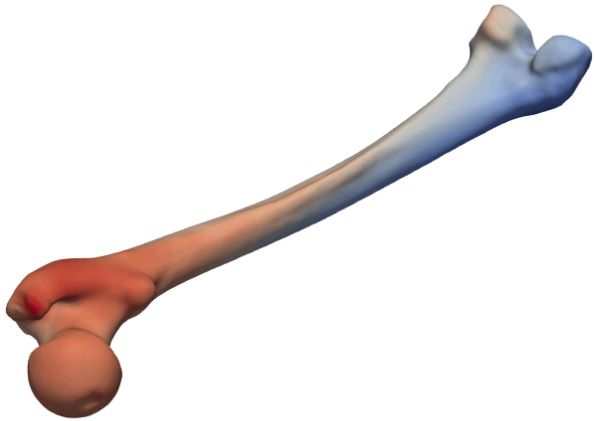


# First visualizations

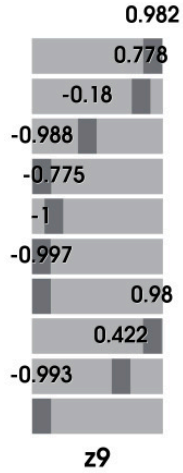
## visuFemur.py



## compare_femur.py



## latent_explorer.py





# PCA on the latent space

Using the latent space we plot the data in 3D using only 3 components.

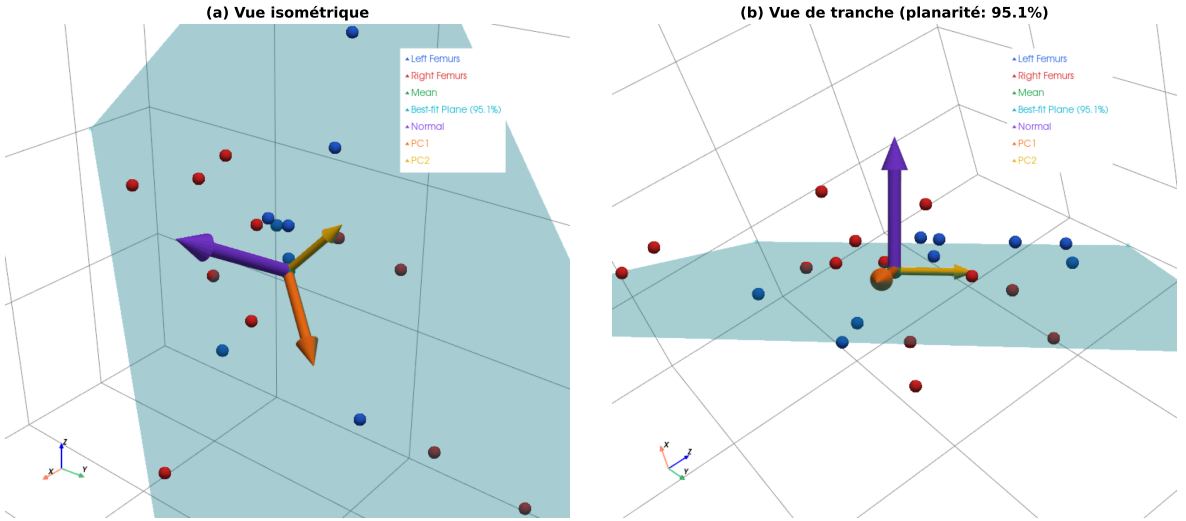



**Figure**: Latent space visualization in 3D

We remark that the data seems organized in a plane.

# PCA on the latent space

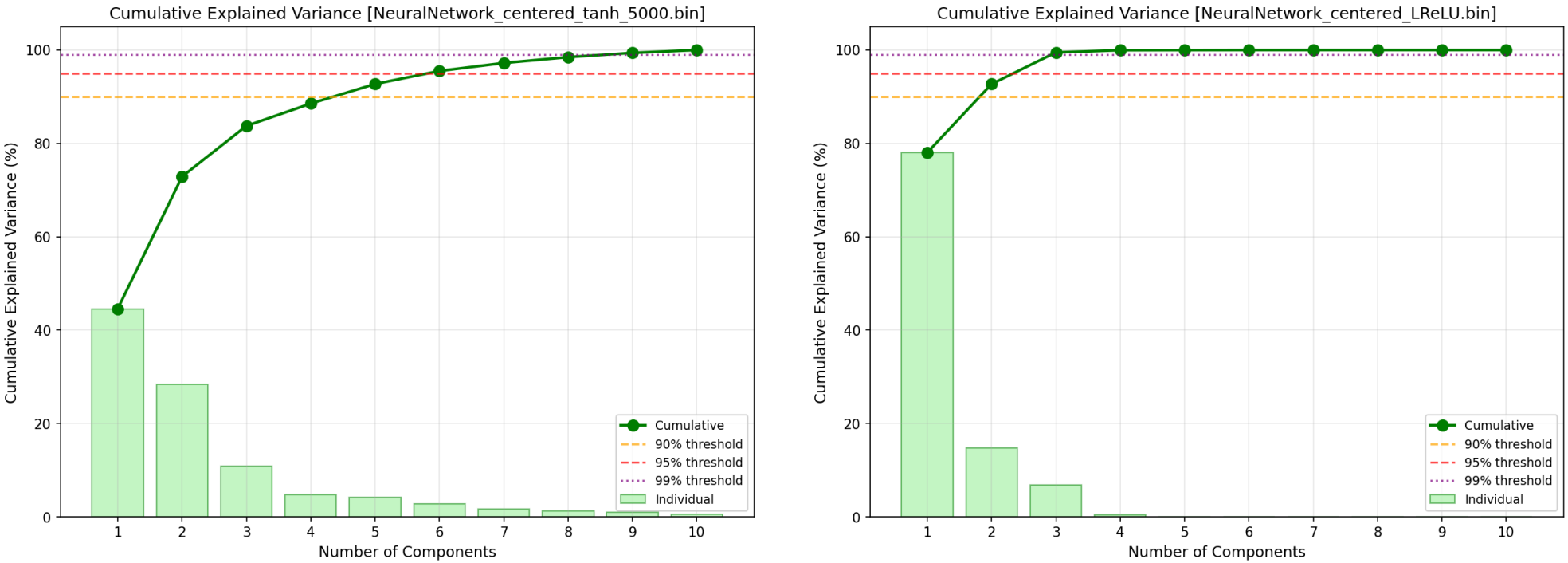



**Figure**: Cumulative variance with PCA on the latent space.

# Results and limitations

## Results

- Neural Network captured **components** of the dataset
- It can be used to **generate** visually plausible femurs
- PCA on the latent space shows that the data is organized in a **subspace** of lower dimension

# Results and limitations

## Results

- Neural Network captured **components** of the dataset
- It can be used to **generate** visually plausible femurs
- PCA on the latent space shows that the data is organized in a **subspace** of lower dimension

## Future work

- **Augment** dataset using PCA
- Increase Neural Network **depth**
- Using **threadpools**
- Train on **unhealthy femurs**
- Implement the **Variational Autoencoder** architecture

# LDDMM : Big Picture

- Instead of treating femurs as vectors in a flat space, we would like to capture the finer underlying structure of the shape space, in order to understand anatomically plausible transformations
- LDDMM learns a representation of the data as a **curved Riemannian manifold** $\mathcal{M}$
- Anatomical deformations are interpreted as following **geodesic** paths on the manifold

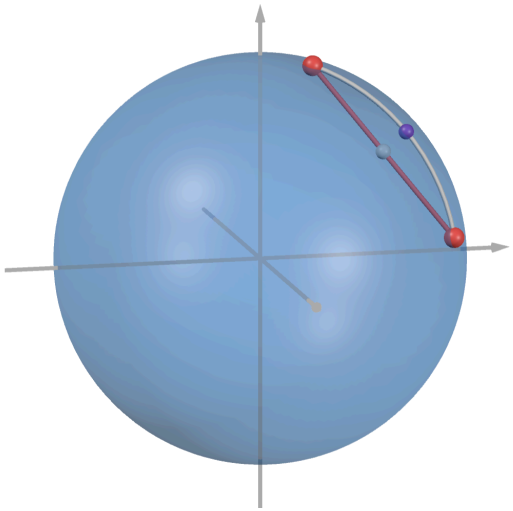


**Figure**: Manifold structure allows for “natural” statistics

# LDDMM : Big Picture

- The space of diffeomorphisms (smooth transformations) over $\mathcal{M}$ is infinite dimensional.
- Based on the **Principle of least action** often respected in nature, we are only interested in the **cheapest** deformations w.r.t some **energy**.
- We define the geodesic distance by minimizing a **variational energy** (i.e cost) :

$$E(v) = \underbrace{\frac{1}{2\sigma_R^2} \int_0^1 \|v_t\|_V^2 \, \mathrm{d}t}_{\text{Regularity}} + \underbrace{\frac{1}{2\sigma_M^2} \sum_i |T_i - \varphi(S_i)|}_{\text{Matching}}$$

- Computationally expensive $\longrightarrow$ need for efficient non-convex optimization algorithms

# LDDMM : Geodesic statistics

- Statistics are computed w.r.t geodesic distance, not euclidean.
  - Guarantee of staying on the manifold and being interpretable as plausible femurs !
- We compute the mean femur (Atlas $\bar{S}$) with respect to the geodesic distance :

$$\bar{S} = \arg\min_{S} \sum_{j=1}^{N} d_G\left(S, S_j\right)^2$$

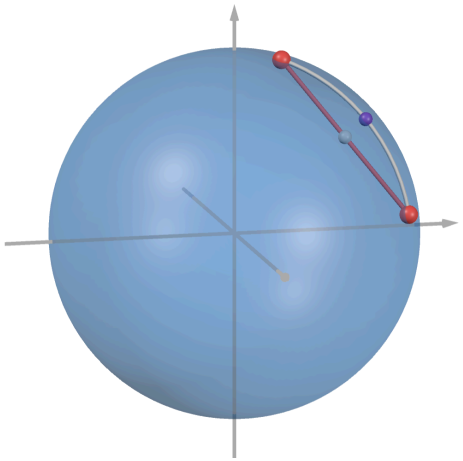


**Figure**: Manifold structure allows for “natural” statistics

# LDDMM : From curved manifold to linear statistics

- **Goal** : Main modes of variation of geodesic deformations.
- **Problem** : The space of geodesic deformations $\mathcal{G}$ is curved so standard linear PCA cannot be applied directly.

# LDDMM : From curved manifold to linear statistics

- **Goal** : Main modes of variation of geodesic deformations.
- **Problem** : The space of geodesic deformations $\mathcal{G}$ is curved so standard linear PCA cannot be applied directly.
- The **geodesic shooting theorem** creates a 1-to-1 map between geodesic deformations of the mean shape and their **initial momenta**
  - This flattens $\mathcal{G}$ into a finite-dimensional vector space $T_{\bar{S}}\mathcal{G} \simeq \mathbb{R}^{3N}$

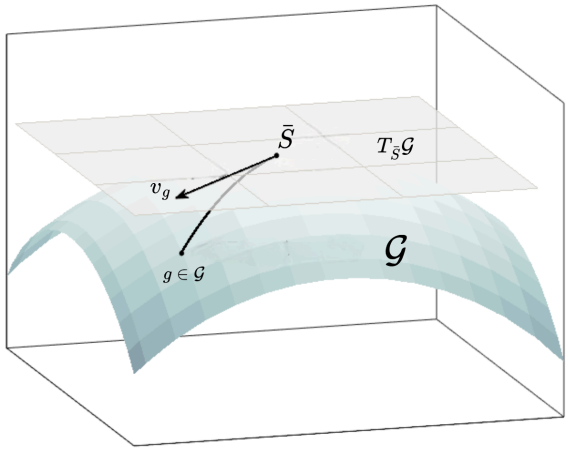



**Figure**: Geodesic shooting linearizes the space of geodesic deformations

# LDDMM : Tangent PCA results and limitations

- This linearization allows us to rigorously apply PCA on $T_{\bar{S}}\mathcal{G}$.
  - Main modes of variation around the mean femur $\bar{S}$
- Deformations are not linear but geodesic
- Accounts for finer **local** deformation compared to Linear PCA which focuses on **global** variation.
- Requires less data than autoencoder, while being more interpretable

# LDDMM : Tangent PCA results and limitations

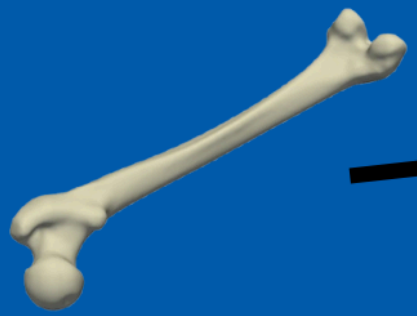
- This linearization allows us to rigorously apply PCA on $T_{\bar{S}}\mathcal{G}$.
  - Main modes of variation around the mean femur $\bar{S}$
- Deformations are not linear but geodesic
- Accounts for finer **local** deformation compared to Linear PCA which focuses on **global** variation.
- Requires less data than autoencoder, while being more interpretable

**Limitations :**

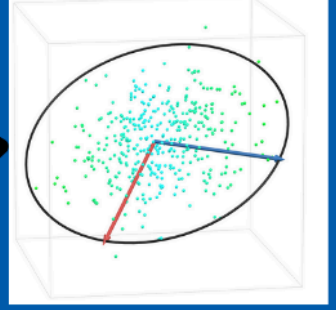
- Very computationally expensive to build the model

**Future work :**

- Recent papers build **diffeomorphic autoencoders** to construct atlases $\longrightarrow$ more efficient.



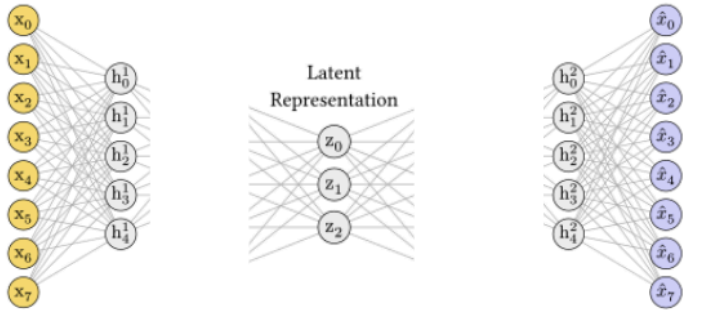
Linear PCA



Linear transformations are not anatomically plausible

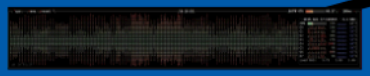
Nonlinear representation

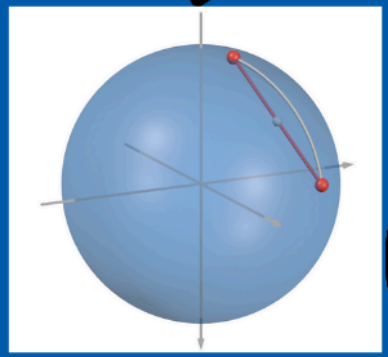
Autoencoder



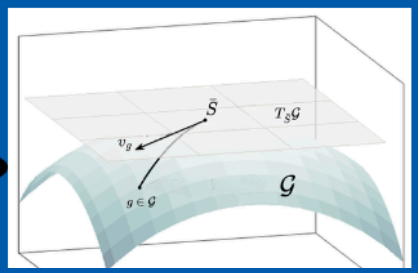

Uninterpretable results + lack of local deformations

Optimizations





Riemannian manifold of Femurs




PCA on geodesic deformations

Anatomically plausible deformations